Summer Linear Attention

A Reference Study of Delta-State Memory with Bounded KV Retrieval

summer

24 October 2026

Technical manuscript draft / Not peer reviewed

Research manuscript draft. This paper specifies and analyzes an untrained NumPy reference implementation. It reports no new benchmark measurements. The evaluation in Section 6 is a proposed, unexecuted protocol. Code scope: SummerLinearAttention v0.1.0, commit 036512212ed0.

Abstract

Fixed-size recurrent memories and explicit key–value stores offer different ways to retain a sequence of associations. A delta-rule state provides a compact linear predictor, whereas a bounded archive preserves selected observations without compressing their values into that state. We describe Summer Linear Attention (SLA), a small reference implementation that combines these components through a single softmax over archived keys and a null candidate. The null candidate assigns its mass to the state prediction rather than to an archived value. The implementation uses normalized keys and queries, fixed update parameters, an insertion-order FIFO archive, and exact archive scanning. We give a code-aligned specification, derive elementary properties of the update and readout, and characterize failures caused by finite softmax sharpness, key collisions, archive occupancy, and eviction. An optional shared-state residual readout is analyzed separately from the default method. With fixed dimensions and archive capacity, the reference has bounded persistent array storage and linear total sequence-processing work; these statements do not imply GPU efficiency or growing-context exact retrieval. Prior work already establishes recurrent-plus-cache hybrids and sentinel-based fallback mixtures. Accordingly, this paper contributes a transparent implementation study and a falsifiable evaluation protocol, not a claim of architectural priority or demonstrated performance superiority.

Keywords: linear attention; delta rule; associative memory; bounded key–value cache; sentinel mixture; reference implementation.

1. Introduction

Sequence models must decide what information to retain and how to retrieve it. Kernel-based linear attention admits recurrent formulations that avoid keeping every previous key–value (KV) pair [1]. Delta-rule fast-weight memories replace purely additive writes with a correction to the current key–value prediction [2]. Efficient sequence-parallel formulations and gated variants have subsequently enabled trained language-model implementations [3, 4]. A compact state, however, does not automatically retain arbitrary associations independently: updates along correlated key directions can interfere.

A bounded explicit archive offers a complementary representation. It can keep a stored value intact even when the recurrent predictor changes, but only while the entry remains in the archive. It also introduces another decision: whether a query should use the recurrent estimate or retrieve an archived observation. SLA isolates these decisions in a small executable specification. Its archive is deliberately simple, and its readout can be inspected without learned projections, controllers, or an end-to-end training system.

The motivating hypothesis is conditional. If values contain a component predictable from keys together with item-specific deviations, a state may summarize part of the structure while an archive retains some exceptions. Whether the proposed mixture actually exploits that complementarity depends on the access distribution, capacity, geometry, and calibration. When values are independent of keys, or several similar keys carry contradictory values, the same design may offer little benefit or introduce error. We do not treat this motivation as an empirical finding.

The contribution of this paper is threefold. First, we specify the exact write, overwrite, eviction, and read semantics of the released reference. Second, we derive implementation-relevant properties and counterexamples, including a distinction between storing an original value and returning that value exactly. Third, we specify a controlled evaluation that separates readout quality, archive policy, and memory budget. No first-of-kind claim is made for combining a recurrent state and a cache, or for allocating softmax mass to a fallback predictor.

2. Related work and scope

Recurrent and delta-rule attention. Katharopoulos et al. [1] formulate linear attention through kernel features and an autoregressive recurrence. Schlag et al. [2] connect linear transformers to fast-weight programming and study delta-rule updates. Yang et al. [3] address sequence-parallel computation of those updates, and Gated Delta Networks [4] combine decay with targeted corrections. SLA uses a scalar-decay delta recurrence; it does not implement those complete architectures or their optimized training algorithms.

Sentinel-based fallback mixtures. Pointer Sentinel Mixture Models [5] jointly normalize pointer scores and a sentinel score, allocating sentinel mass to a vocabulary predictor. SLA applies a related structural idea to continuous values: a fixed scalar null logit receives mass for the delta-state prediction. The fallback-mixture principle therefore predates this reference. The output space and parameterization differ, not the general idea of a softmax fallback.

Bounded caches beside recurrent states. HOLA [6] is particularly close: it combines delta-state memory with a bounded original-KV cache and a null sink. It uses residual-based retention, a sharpened cache path, and additive gated integration. SLA instead exposes FIFO retention and a joint normalized mixture whose null mass weights the state prediction. These differences define the object studied here, not an established improvement. Learnable Token Eviction [7] also studies bounded retained KV pairs in hybrid linear/sparse-attention models; SLA has no learned eviction module.

Companion SDA. Summer Delta Attention (SDA) is a separate companion project [9]. The SLA documentation distinguishes its null-aware soft mixture from SDA's hard, confidence-gated top-1 recall. The optional residual extension below is still a soft mixture, not an implementation of that hard selection rule. Results, memory numbers, or figures from SDA cannot be relabeled as SLA results.

Our subject is the public `SLAMemory` class at the pinned SLA commit [8]. The repository identifies the author as summer and licenses its code under MIT. It contains no trained model, learned key/query projection, automatic differentiation pipeline, retrieval index, or optimized GPU kernel. This is a method/reference study rather than a benchmark paper or a comprehensive review of hybrid attention.

3. Method

3.1. Objects and conventions

Let d_k and d_v be positive key and value dimensions, and let $B \geq 0$ be an integer archive capacity. The memory consists of a matrix $S \in \mathbb{R}^{d_k \times d_v}$, initialized to zero, and an ordered archive

$$\mathcal{A} = [(\kappa_i, u_i, \iota_i)]_{i=1}^{m}, \qquad 0 \leq m \leq B.$$

Here κ_i is a unit-norm key, u_i a stored value, and ι_i an optional integer identity used only by writes. Reads do not receive an identity. Every nonzero finite input key and query is normalized to Euclidean norm one. Values are copied after conversion to the chosen floating-point type. Thus "original KV" means normalized keys and stored, dtype-converted values, not the unnormalized input key or arbitrary-precision data.

Parameters are scalar decay $\alpha \in [0, 1]$, write strength $\beta \in (0, 1]$, score scale $s > 0$, and finite null logit c. The reference defaults are $B = 32$, $\alpha = 1$, $\beta = 0.05$, $s = 32$, $c = 0$, and float64. Float32 is also accepted. All these parameters are fixed configuration values, not learned gates.

3.2. Delta-state write

For a normalized key k_t and value v_t, the update is

$$\bar{S}_t = \alpha S_{t-1}, \qquad e_t = v_t - \bar{S}_t^\top k_t, \qquad S_t = \bar{S}_t + \beta k_t e_t^\top. \tag{1}$$

The prediction residual is computed after decay. Every valid write updates the state, including a repeated identity and a write with $B = 0$. An archive overwrite does not undo an earlier state's contribution. The API exposes writes and reads separately: a read sees all writes completed before it. A sequence experiment must therefore state whether its evaluation is before or after the current write, and must not write a withheld answer before predicting it.

3.3. Archive writes and eviction

After the state update, an archive write follows these rules:

1. If $B = 0$, leave the archive empty.
2. If a non-null integer identity matches a retained entry, replace that entry's key and value in place. Its FIFO position is unchanged.
3. Otherwise, append a new entry if capacity remains.
4. If full, remove the earliest inserted retained entry, shift the remaining entries, and append the new entry.

An unspecified identity never matches an existing entry. Distinct identities can share an identical normalized key. Consequently, this policy is FIFO over insertion slots, not least-recently-used replacement and not a sliding window over the most recent write operations when in-place overwrites occur. There is no surprise threshold, utility ranking, learned admission gate, or query-triggered archive write.

3.4. Null-aware readout

Given unit query q, first compute the fast-state prediction $f(q) = S^\top q$. For a nonempty archive, define

$$\ell_0 = c, \qquad \ell_i = s\kappa_i^\top q, \qquad w_i = \frac{\exp(\ell_i)}{\sum_{j=0}^{m} \exp(\ell_j)}. \tag{2}$$

The default SLA readout is

$$y_0(q) = w_0 f(q) + \sum_{i=1}^{m} w_i u_i. \tag{3}$$

The implementation subtracts the maximum logit before exponentiation. With an empty archive, it directly returns $f(q)$ and reports null weight one. The null candidate is therefore a route to the state, not an output-zero token or an explicit "answer absent" decision. Its weight is not a calibrated probability of correctness or missingness.

3.5. Optional shared-state residual readout

The API also exposes a fixed read-time coefficient $\lambda \in [0, 1]$:

$$y_\lambda(q) = y_0(q) + \lambda \sum_{i=1}^{m} w_i S^\top (q - \kappa_i). \tag{4}$$

All terms use the current state, including predictions at old archived keys. Equation (4) can be rearranged as

$$y_\lambda(q) = \{w_0 + \lambda(1 - w_0)\} f(q) + \sum_{i=1}^{m} w_i \{u_i - \lambda S^\top \kappa_i\}. \tag{5}$$

At $\lambda = 0$, this is ordinary SLA. At $\lambda = 1$, it is $f(q) + \sum_i w_i(u_i - S^\top \kappa_i)$, a state predictor plus soft residual corrections. It need not lie in the convex hull of $f(q)$ and the stored values. It neither selects a single archived entry nor guarantees exact recall, and adaptive selection of λ is outside the released reference.

3.6. Executable specification

Algorithm 1: SLA write and read. The following pseudocode mirrors the reference, omitting only input checks and dtype casts.

```
WRITE(key, value, optional id):
    k = unit(key); v = copy(value)
    S = alpha * S
    S = S + beta * outer(k, v - transpose(S) * k)
    if B == 0: return
    if id is supplied and retained:
        replace that slot's key and value; return
    if archive is full: remove first slot
    append (k, v, id)

READ(query, lambda = 0):
    q = unit(query); f = transpose(S) * q
    if archive is empty: return f
    w = softmax([c, s * dot(k_i, q) for each slot i])
    y = w[0] * f + sum_i w[i+1] * v_i
    y = y + lambda * sum_i w[i+1] * transpose(S) * (q-k_i)
    return y
```

4. Analytical properties and failure modes

The statements in this section concern exact real arithmetic. They are mathematical consequences of Equations (1)–(4), not measured performance or numerical-stability guarantees.

4.1. A write contracts the residual at its own key

Proposition 1. For a unit key, the post-write residual obeys

$$v_t - S_t^\top k_t = (1-\beta)e_t. \tag{6}$$

Proof. Multiply Equation (1) by k_t and use $k_t^\top k_t = 1$. The new prediction is $\bar{S}_t^\top k_t + \beta e_t$, giving Equation (6). Thus a write with $\beta = 1$ fits the current pair exactly in the state immediately after that write. It does not preserve every previous pair. For another unit key r, the write changes the decayed prediction by $\beta(k_t^\top r)e_t$, making correlation-dependent interference explicit.

4.2. A conditional bound on state magnitude

Rewrite the update as

$$S_t = \alpha(I - \beta k_t k_t^\top)S_{t-1} + \beta k_t v_t^\top. \tag{7}$$

The matrix $I - \beta k_t k_t^\top$ has spectral norm at most one for the allowed β. If $\|v_t\|_2 \le V$ and $S_0 = 0$, the Frobenius norm satisfies

$$\|S_t\|_F \le \begin{cases} \beta V(1-\alpha^t)/(1-\alpha), & 0 \le \alpha < 1, \\ t\beta V, & \alpha = 1. \end{cases} \tag{8}$$

This follows from the triangle inequality and iteration of $\|S_t\|_F \le \alpha\|S_{t-1}\|_F + \beta V$. The second line is only a finite-horizon upper bound, not a proof of time-uniform boundedness at the default $\alpha = 1$. Normalizing keys does not by itself bound arbitrary input values or prevent floating-point overflow.

4.3. Retrieval depends on margin, null competition, and values

For $\lambda = 0$, Equation (3) is a convex combination of the current state prediction and archived values. For a target archived value u_j,

$$\|y_0 - u_j\|_2 \le w_0 \|f - u_j\|_2 + \sum_{i \neq j} w_i \|u_i - u_j\|_2. \tag{9}$$

Suppose the target's cosine similarity exceeds every other archived similarity by at least $\delta > 0$. Then

$$w_j \ge \left[1 + e^{c - s\kappa_j^\top q} + (m-1)e^{-s\delta}\right]^{-1}. \tag{10}$$

To prove this, divide the softmax denominator by $\exp(s\kappa_j^\top q)$ and bound each competitor's ratio by $e^{-s\delta}$. The bound isolates three requirements for concentrated retrieval: a sufficient key margin, control of the number of competitors, and dominance over the null logit. Even when these hold, Equation (9) still depends on the magnitudes of competing values and state error. A correct top-1 key alone is insufficient.

4.4. Original storage is not exact readout

Consider a zero-initialized state, one write (k, v) with $v \neq 0$, an archive containing that one pair, and query $q = k$. Equation (1) gives $f = \beta v$. For any allowed λ, the residual correction vanishes because $q - k = 0$, and

$$y_\lambda = [1 - w_0(1 - \beta)]v. \tag{11}$$

For finite logits, $w_0 > 0$ in exact arithmetic. Hence $\beta < 1$ prevents exact output even though the archived key matches perfectly and its value is intact. This is a symbolic counterexample, not an observed benchmark result.

Soft averaging also fails under contradictory near-neighbors. Let two stored values be u and $-u$, with query similarities 1 and $1 - \delta$. Their combined archive contribution is $(w_+ - w_-)u$. As $\delta \to 0$, it tends to zero, although the positive-value key is the unique top-1 match for every $\delta > 0$. At identical keys, identity labels cannot resolve the ambiguity because the read interface does not accept them.

4.5. Archive occupancy changes fallback behavior

If all m archive entries have the same score ℓ, then

$$w_0 = \frac{1}{1 + m\exp(\ell - c)}. \tag{12}$$

Adding equally scored candidates therefore reduces state weight without improving any individual match. A null logit calibrated at one capacity or occupancy need not transfer to another. It is a routing parameter, not a missingness certificate. The reference contains no occupancy correction.

Finally, Equation (4) gives the simple perturbation bound

$$\|y_\lambda - y_0\|_2 \le 2\lambda(1 - w_0)\|S\|_2, \tag{13}$$

because both queries and keys have unit norm. The bound does not say whether the correction helps. It shows why residual mixing must be evaluated separately rather than assumed to improve the default readout.

5. Computational and implementation accounting

For a dtype using b bytes per scalar, the preallocated state, key, and value arrays occupy exactly

$$M_{\text{arrays}} = b\{d_k d_v + B(d_k + d_v)\}. \tag{14}$$

This is what `allocated_array_bytes` measures. It excludes Python objects, identity storage, allocator overhead, temporary arrays, and the surrounding program. It counts full allocated capacity even when the archive is partly empty. It is not process memory, peak resident memory, or the total cache footprint of a language model.

One state update costs $O(d_k d_v)$. An exact read costs $O(d_k d_v + B(d_k + d_v))$ in the worst case. The implementation evaluates the optional residual through an aggregated key and matrix multiplication, preserving that asymptotic cost rather than performing B separate state projections. A full-archive write may also cost $O(B(d_k + d_v))$ for array shifts and $O(B)$ for identity lookup. A ring buffer could reduce movement but is not the shipped implementation.

With one write and one read per step, fixed B, d_k, d_v yield $O(N)$ work over N steps and persistent array storage independent of N. If B scales proportionally to N, repeated exact scans may instead require $O(N^2)$ work. Thus the name "linear attention" refers here to the fixed-capacity sequence-cost regime. The entire readout is not linear in its query, and no claim is made that this NumPy implementation is faster than a particular attention kernel.

Inputs are checked before ordinary write mutations; zero or non-finite keys/queries and invalid value shapes are rejected. Key normalization first rescales by the largest absolute component, reducing overflow risk during the norm calculation. These safeguards do not constitute exhaustive numerical validation. For example, finite input values can still produce non-finite intermediate state arithmetic. The reference includes functional tests, but a test suite is not evidence of accuracy, throughput, or trained-model behavior.

6. Proposed evaluation protocol: not executed

No experiment in this section has been run for this manuscript. The pinned repository discusses earlier exploratory observations but does not provide their raw benchmark tables, complete configuration-to-result trail, or figure-generation package. We therefore exclude those observations from the evidential results of this paper. The following protocol describes what would be required to test the method.

6.1. Questions and controlled tasks

The primary question is whether SLA improves held-out reconstruction error over both its state-only component and an archive-only method under the same memory budget. A secondary question is which conditions cause losses. Use a factorial synthetic suite rather than one favorable aggregate:

- **Independent associations:** unit-normalized random keys and independently drawn values, to test memorization without an available linear mapping.
- **Structured values with exceptions:** $v_i = A^\top k_i + \epsilon_i + z_i r_i$, with matrix, noise, exception rate, and exception magnitude fixed by a published generator. This tests the motivating hypothesis rather than assuming it.
- **Query perturbations:** $q = \text{unit}(k_i + \sigma\xi)$ with explicit distributions and both calibration-range and out-of-range σ.

- **Recency and access shifts:** uniform, recent-biased, and recurring-key queries, including a change in the access distribution after calibration.
- **Collisions and abstention stress:** near-duplicate keys with contradictory values, exact duplicate keys with distinct identities, and queries whose target entry is not retained.

Report retained-target and evicted-target cases separately. "Absent from the archive" and "never observed by the state" are different conditions. Define the target convention for repeated identities and report overwrite and insertion ages separately. For unobserved queries, define a target-generating map before evaluating error; do not call every null-routed output a correct abstention.

6.2. Baselines and memory fairness

Compare state-only delta memory; FIFO KV retrieval without a state; FIFO KV with a separately calibrated null/zero fallback; default SLA; and the optional λ variants. Add hard top-1 retrieval with an explicit fallback and a separately pinned SDA implementation when its exact configuration is available. Full-history KV retrieval can serve as an unconstrained-memory reference, but not as a same-budget comparator. A full trained Transformer, GDN, or Kimi Delta Attention (KDA) model [10] is a different experimental tier and must not be ranked from these untrained memory tests.

Two fairness regimes should be published. **Component-controlled** comparisons hold the archive contents, capacity, state, and query stream fixed to isolate readout changes. **Budget-controlled** comparisons allow each method to spend the same total memory. Under array-payload accounting alone, an archive-only method can use

$$B_{\mathrm{KV}} = \left\lfloor \frac{d_k d_v + B(d_k + d_v)}{d_k + d_v} \right\rfloor \tag{15}$$

entries at the same dtype. Metadata and runtime temporaries require separate measurement. Reporting only a same-B comparison would hide the extra state budget.

6.3. Calibration, statistics, and reporting

Publish disjoint calibration and test seeds, generator versions, and the entire hyperparameter search. Tune s, c, α, β, and any hard-retrieval threshold on calibration data only, with comparable search budgets for competing methods. Evaluate $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$ as predefined ablations, not a test-set oracle. Include occupancy and capacity sweeps so that Equation (12) is tested directly.

For every condition, report mean squared reconstruction error, an explicitly normalized error, and recall under a predefined error tolerance. State denominators and handling of near-zero target energy. Record archive retention rate, null-weight distribution, and performance by target age. Report paired differences on shared test streams with seed-level confidence intervals, using at least 32 independent test seeds as a proposed starting design. Queries from one stream should not be treated as independent experimental replicates. Publish each condition, not only a macro-average, and identify exploratory analyses and multiple-comparison handling.

6.4. Systems measurements and later model evaluation

Measure writes and reads separately, then end-to-end streams, after warmup. Report hardware, software versions, dtype, thread settings, dimensions, capacity, occupancy, and repeated timing distributions. Distinguish persistent arrays, peak process memory, and temporary allocations. A CPU NumPy timing cannot establish GPU training or decoding efficiency.

Only a subsequent trained-model study can support language-model claims. Such a study would need a specified architecture, causal integration, differentiability treatment, parameter and training-token budgets, datasets and licenses, optimization settings, multiple training seeds, and appropriate trained baselines. HOLA and other close cache-augmented systems must be included or their omission justified. None of these experiments is supplied by the current reference.

7. Limitations and responsible interpretation

The archive has finite retention and does not recover an evicted observation exactly unless another component happens to predict it. FIFO is easy to reproduce but is not universally appropriate. In-place identity overwrites preserve old eviction positions, so recent updates can disappear sooner than a reader expecting least-recently-used semantics would assume. Keys that collide cannot be disambiguated by identity at read time.

The null candidate introduces a fallback route but not semantic abstention. Finite softmax mixing can attenuate exact matches or blend contradictory values. Neither the score scale nor residual mixing has a distribution-independent optimum. State decay changes the persistence of information, and its interaction with archive retention remains an empirical question.

The implementation handles a single memory instance, without a defined multi-head model, batching strategy, positional mechanism, training objective, or production serving policy. Its simplicity is useful for inspection but limits external validity. The analytical results are intentionally elementary and local; they do not establish a generalization theorem, optimal archive policy, superiority over prior methods, or end-to-end numerical stability.

Explicit caches can retain sensitive values verbatim. The reference provides no privacy guarantee, deletion protocol for historical state contributions, or secure memory lifecycle. A deployment would require application-specific retention and access controls. These are not addressed by the method's asymptotic memory bound.

8. Conclusion

SLA offers a compact, code-aligned example of a delta-state predictor combined with a bounded original-KV archive and a null-aware soft readout. Its value at this stage is inspectability: storage, routing, overwrite semantics, and failure modes can be stated precisely. The analysis shows both the local corrective behavior of delta writes and the limits of soft retrieval, including the failure of exact recall even in a one-entry example. The next scientific step is the controlled, budget-aware evaluation specified here. Until that evidence exists, SLA should be presented as an experimental reference, not as a demonstrated replacement for trained attention architectures.

Appendix A. Artifact and reproducibility statement

The source of truth is SummerLinearAttention at commit 036512212ed0. The exact full identifier is `036512212ed0f7d2456988cb4b523599064a7e6f`. The implementation is in reference.py; the package metadata specifies Python 3.10 or later and NumPy 1.24 or later. The code license is MIT.

The repository documents installation, functional checks, and an example:

```
python -m pip install -e .
python -m unittest discover -s tests -v
python -m examples.quickstart
```

The committed tests cover zero capacity, same-identity overwrite, distinct identities sharing a key, FIFO eviction, an invalid write, large-magnitude normalization, lambda-zero equivalence, array-byte

accounting, and a single-key residual case. Listing these tests is not a claim that they were executed for this paper. Before submission, archive execution logs and an environment manifest. Before adding empirical results, publish the generator, calibration/test split, raw per-seed outputs, analysis code, plotting code, and checks linking every reported number to its artifact.

References

[1] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. ICML, PMLR 119, 5156–5165. Publisher record.

[2] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. 2021. Linear Transformers Are Secretly Fast Weight Programmers. ICML, PMLR 139, 9355–9366. Publisher record.

[3] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. 2024. Parallelizing Linear Transformers with the Delta Rule over Sequence Length. arXiv:2406.06484.

[4] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2025. Gated Delta Networks: Improving Mamba2 with Delta Rule. ICLR. Official publication record; arXiv:2412.06464.

[5] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. arXiv:1609.07843.

[6] Wanyun Cui. 2026. A Hippocampus for Linear Attention: An Exact Memory for What the Recurrent State Forgets. arXiv:2607.02303v1.

[7] Mutian He and Philip N. Garner. 2025. Alleviating Forgetfulness of Linear Attention by Hybrid Sparse Attention and Contextualized Learnable Token Eviction. arXiv:2510.20787v2.

[8] summer. 2026. Summer Linear Attention: NumPy Reference Implementation, v0.1.0. Pinned source repository. MIT license. Accessed 24 October 2026.

[9] summer. Summer Delta Attention. Companion project. Repository. Cited for project context, not as empirical evidence for SLA. Accessed 24 October 2026.

[10] Kimi Team. 2025. Kimi Linear: An Expressive, Efficient Attention Architecture. arXiv:2510.26692v2.